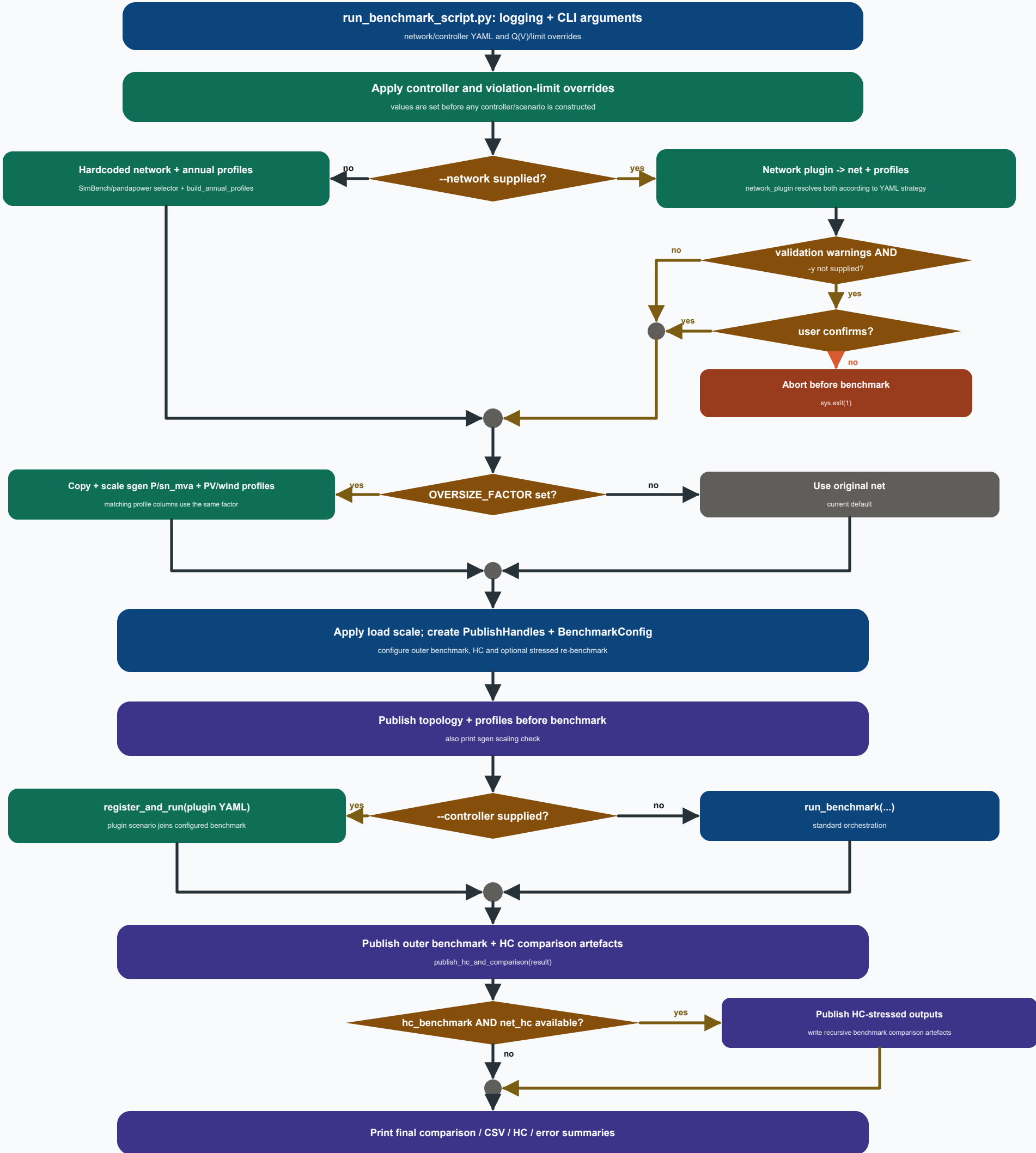




Entry configuration

The direct script configures logging, parses optional network/controller YAML paths and accepts runtime Q(V) and planning-limit overrides.

Q(V) breakpoints and q-ratio are applied before any controller is constructed, so software and Arduino configuration use the same values.

Voltage and thermal limits are updated in violation_detector before benchmark dispatch.

The script creates separate publisher handles for the outer network and an optional HC-stressed network.

BenchmarkConfig carries scenario selection, dry-run/serial settings, HC controls, profile factory, checkpointing and publishing handles.

Network and profile resolution

With --network, network_plugin loads both the network and profiles according to the YAML strategy, then validation warnings are reported.

If validation warnings require confirmation and -y is absent, the user can abort before any benchmark execution.

Without --network, the selected built-in network is loaded and build_annual_profiles creates the annual profile set.

Generation oversizing is conditional; sgen P/sn_mva and matching PV/wind profile columns use the same scale factor.

LOAD_SCALE is then applied consistently to network loads and the load profile before topology/profile publication.

Benchmark dispatch and outputs

Topology and profiles are published once before the benchmark so the saved model corresponds to the network entering orchestration.

With --controller, plugin_runner temporarily registers the custom controller and runs it alongside BenchmarkConfig.scenarios.

Without --controller, the script calls benchmark_runner.run_benchmark directly.

After execution, publish_hc_and_comparison writes the outer comparison/HC artefacts.

If hc_benchmark and net_hc are available, the recursive benchmark artefacts are written to the separate HC-stressed output directory.

The script finally prints comparison, CSV, hosting-capacity, error and HC-stressed summaries when those result sections are available.